SISTEMAS DISTRIBUÍDOS · TRABALHO 2

Controle de gastos em arquitetura de microsserviços

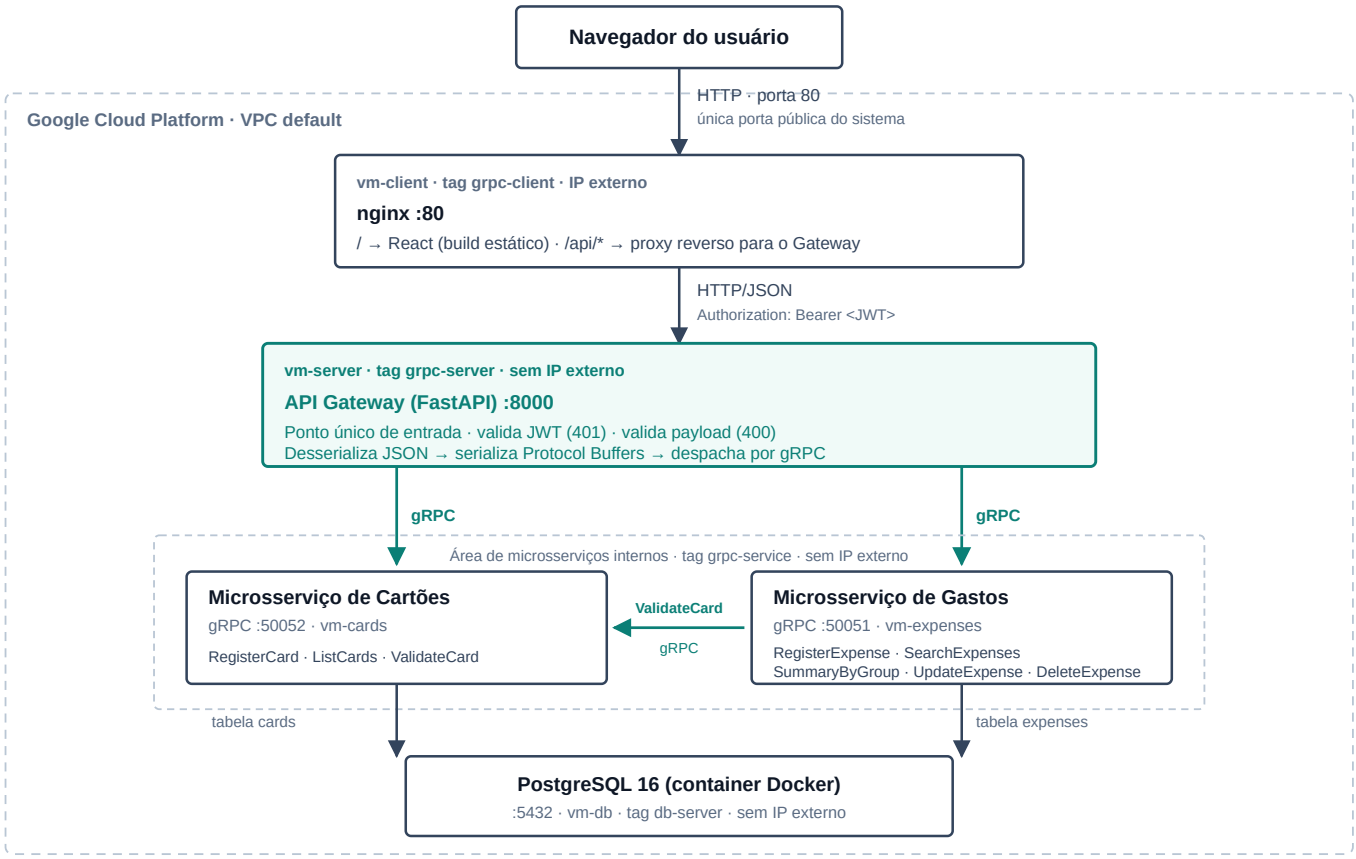
Frontend web, API Gateway REST e dois microsserviços gRPC sobre PostgreSQL

INTEGRANTES	APRESENTAÇÃO	NUVEM	INFRAESTRUTURA
Murilo Rodrigues · Caio Oliveira	**22/10/2026 · Lab 154**	**Google Cloud Platform**	**5 VMs, uma por serviço**

Código-fonte: **github.com/murilorrs/sd-gastos-grpc**

Frontend em `web/` · API Gateway em `gateway/` · microsserviços em `server/` · contrato em `proto/expenses.proto`

Arquitetura de microsserviços implementada



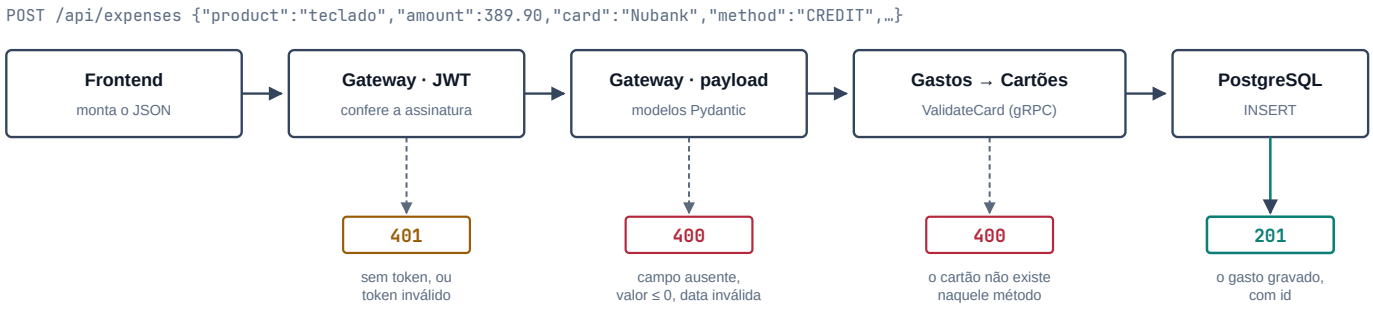
Cinco VMs no GCP, uma por serviço. Nenhuma seta liga o navegador diretamente aos microsserviços, e nenhuma liga o Gateway ao banco: cada peça só alcança a vizinha. As setas em destaque são as chamadas gRPC com Protocol Buffers.

Componentes

COMPONENTE	TECNOLOGIA	RESPONSABILIDADE
Frontend `web/`	React 19, TypeScript, Vite servido por nginx	Interface visual: painel com gráficos, cadastro e edição de gastos e cartões, assistente em linguagem natural e tela de diagnóstico. Comunica-se **exclusivamente** com o API Gateway, pelo caminho relativo `/api`.
API Gateway `gateway/app.py`	FastAPI (Python) Pydantic, PyJWT, grpcio	Ponto único de entrada. Autentica o JWT, valida o payload, traduz JSON em Protocol Buffers e orquestra as chamadas aos microsserviços. Não acessa o banco.
Microsserviço de Cartões `server/cards_service.py`	Python, grpcio, psycopg	Dono da tabela `cards`. Cadastra, lista e valida cartões, com correção de grafia por similaridade.
Microsserviço de Gastos `server/expenses_service.py`	Python, grpcio, psycopg	Dono da tabela `expenses`. Grava, busca, soma, altera e remove. **Consulta o microsserviço de Cartões por gRPC** antes de gravar ou alterar.
Banco de dados `vm-db`	PostgreSQL 16 em container	Persistência real. Todas as operações de consulta, inserção, alteração e remoção passam por ele.

Fluxo de uma requisição

O registro de um gasto atravessa os cinco componentes. Cada etapa pode interromper o fluxo com um status HTTP específico:



As três saídas tracejadas interrompem o fluxo antes do banco. O `401` ocorre antes de qualquer canal gRPC ser aberto; o primeiro `400` é validação de formato, e o segundo é validação de negócio, decidida pelo microsserviço de Cartões.

Contrato gRPC

O arquivo `proto/expenses.proto` define dois serviços e oito RPCs. Os stubs não são versionados: são gerados a partir do contrato em cada instalação, o que impede divergência entre as pontas.

SERVIÇO	RPC	TIPO	FINALIDADE
CardService `:50052`	`RegisterCard`	unário	Cadastra a dupla (nome, método de pagamento).
	`ListCards`	unário	Lista os cartões cadastrados.
	`ValidateCard`	unário	Confere um cartão e sugere nomes parecidos em caso de erro de digitação.
ExpenseService `:50051`	`RegisterExpense`	unário	Grava o gasto, após consultar o CardService.
	`SearchExpenses`	**server-streaming**	Devolve os gastos um a um, conforme os lê do banco.
	`SummaryByGroup`	unário	Totais agrupados por categoria, cartão ou método.
	`UpdateExpense`	unário	Altera o gasto, revalidando o cartão.
	`DeleteExpense`	unário	Remove o gasto.

Segurança de rede

Os microsserviços escutam em `0.0.0.0` por terem VMs próprias. O isolamento é garantido por pares de regras de firewall, avaliadas por prioridade — no GCP, o menor número é avaliado primeiro:

PRIORIDADE	AÇÃO	PORTA	ORIGEM	DESTINO
1000	permitir	`tcp:80`	`0.0.0.0/0`	`grpc-client`
1000	permitir	`tcp:8000`	`10.128.0.0/9` (VPC)	`grpc-server`
800	permitir	`tcp:50051,50052`	tags `grpc-server`, `grpc-service`	`grpc-service`
900	**negar**	`tcp:50051,50052`	`0.0.0.0/0`	`grpc-service`
800	permitir	`tcp:5432`	tag `grpc-service`	`db-server`
900	**negar**	`tcp:5432`	`0.0.0.0/0`	`db-server`

> A regra `default-allow-internal`, criada automaticamente pelo GCP em toda VPC, libera todas as portas entre VMs e tem prioridade 65534 — nunca é alcançada para essas portas. Sem as regras de negação, a VM do frontend conseguiria falar diretamente com os microsserviços, violando o requisito de que o frontend se comunique apenas com o Gateway.

Atendimento aos requisitos

REQUISITO	IMPLEMENTAÇÃO
Interface visual funcional	`web/` — React + TypeScript, cinco telas
Frontend fala apenas com o Gateway	Todas as chamadas vão para `/api`; não há endereço de microsserviço no código do frontend, e o firewall o impediria de alcançá-los
API Gateway REST, ponto único de entrada	`gateway/app.py` — FastAPI
Mínimo de 2 microsserviços via gRPC	`server/cards_service.py` e `server/expenses_service.py`; contrato em `proto/expenses.proto`
Comunicação entre microsserviços	`ExpenseService._validate_card` chama `CardService.ValidateCard` antes de gravar e de alterar
Banco de dados real, sem mocks	PostgreSQL 16; consulta, inserção, alteração e remoção passam por ele. Não há lista em memória no projeto
Validação com `400` e `201`	Modelos Pydantic no Gateway; tratador converte o 422 padrão do FastAPI em 400
Autenticação JWT com `401`	Dependência `require_token` aplicada ao roteador inteiro de negócio
Tradução JSON → Protobuf → JSON	Função `call()` do Gateway, que registra em log o tamanho de cada mensagem protobuf

Documentação completa no repositório: `README.md` (arquitetura e API), `docs/DEPLOY.md` (provisionamento automatizado) e `docs/GCP-CONSOLE.md` (passo a passo pelo console).